```
11.
12.  public static void main(String args[]){
13.  TestInterruptingThread1 t1=new TestInterruptingThread1();
14.  t1.start();
15.  try{
16.  t1.interrupt();
17.  }catch(Exception e){System.out.println("Exception handled "+e);}
18.18.
19.  }
20.  }
```

Output:Exception in thread-0
java.lang.RuntimeException: Thread interrupted...
java.lang.InterruptedException: sleep interrupted at A.run(A.java:7)

Example of interrupting a thread that doesn't stop working

In this example, after interrupting the thread, we handle the exception, so it will break out the sleeping but will not stop working.

```
1.  class TestInterruptingThread2 extends Thread{
2.  public void run(){
3.  try{
4.  Thread.sleep(1000);
5.  System.out.println("task");
6.  }catch(InterruptedException e){
7.  System.out.println("Exception handled
"+e); 8. }
9. System.out.println("thread is running...");
10. }
11.
12.  public static void main(String args[]){
13.  TestInterruptingThread2 t1=new TestInterruptingThread2();
14.  t1.start();
15.
16. t1.interrupt();
17.
18.  }
19.  }
```

Output:Exception handled
java.lang.InterruptedException: sleep interrupted thread is running...

Example of interrupting thread that behaves normally

If thread is not in sleeping or waiting state, calling the interrupt() method sets the interrupted flag to true that can be used to stop the thread by the java programmer later.

```
1. class TestInterruptingThread3 extends Thread{
2.
3. public void run(){
4. for(int i=1;i<=5;i++)
5. System.out.println(i);
6. }
```

```
7.
8.  public static void main(String args[]){
9.  TestInterruptingThread3 t1=new TestInterruptingThread3();
10. t1.start();
11.
12. t1.interrupt();
13.
14. }
15. }
```

Output:1
2
3
4
5

What About Isinterrupted And Interrupted Method?

The isInterrupted() method returns the interrupted flag either true or false. The static interrupted() method returns the interrupted flag afterthat it sets the flag to false if it is true.

```
1. public class TestInterruptingThread4 extends Thread{
2.
3. public void run(){
4. for(int i=1;i<=2;i++){
5.  if(Thread.interrupted()){
6.  System.out.println("code for interrupted
thread"); 7. }
8.  else{
9.  System.out.println("code for normal thread");
10. }
11.
12. }//end of for loop
13. }
14.
15. public static void main(String args[]){
16.
17. TestInterruptingThread4 t1=new TestInterruptingThread4();
18. TestInterruptingThread4 t2=new TestInterruptingThread4();
19.
20. t1.start();
21. t1.interrupt();
22.
23. t2.start();
24.
25. }
26. }
```

Output:Code for interrupted thread code for normal
thread
code for normal thread code for
normal thread

UNIT-1V

Collection Framework in java –Introduction to java collections- overview of java collection frame work-generics-commonly used collection classes- Array List- vector -hash table-stack-enumeration-iterator-string tokenizer -random -scanner -calendar and properties

Files – streams–byte streams- character stream- text input/output- binary input/output- file management using file class.

Connecting to Database –JDBC Type 1 to 4 drivers- connecting to a database- quering a database and processing the results- updating data with JDBC Data Access Object(DAO)

Collections in Java

Collections in java is a framework that provides an architecture to store and manipulate the group of objects.

All the operations that you perform on a data such as searching, sorting, insertion, manipulation, deletion etc. can be performed by Java Collections.

Java Collection simply means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque etc.) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet etc).

WHAT IS COLLECTION IN JAVA

Collection represents a single unit of objects i.e. a group.

What is framework in java

- provides readymade architecture.
- represents set of classes and interface.
- is optional.

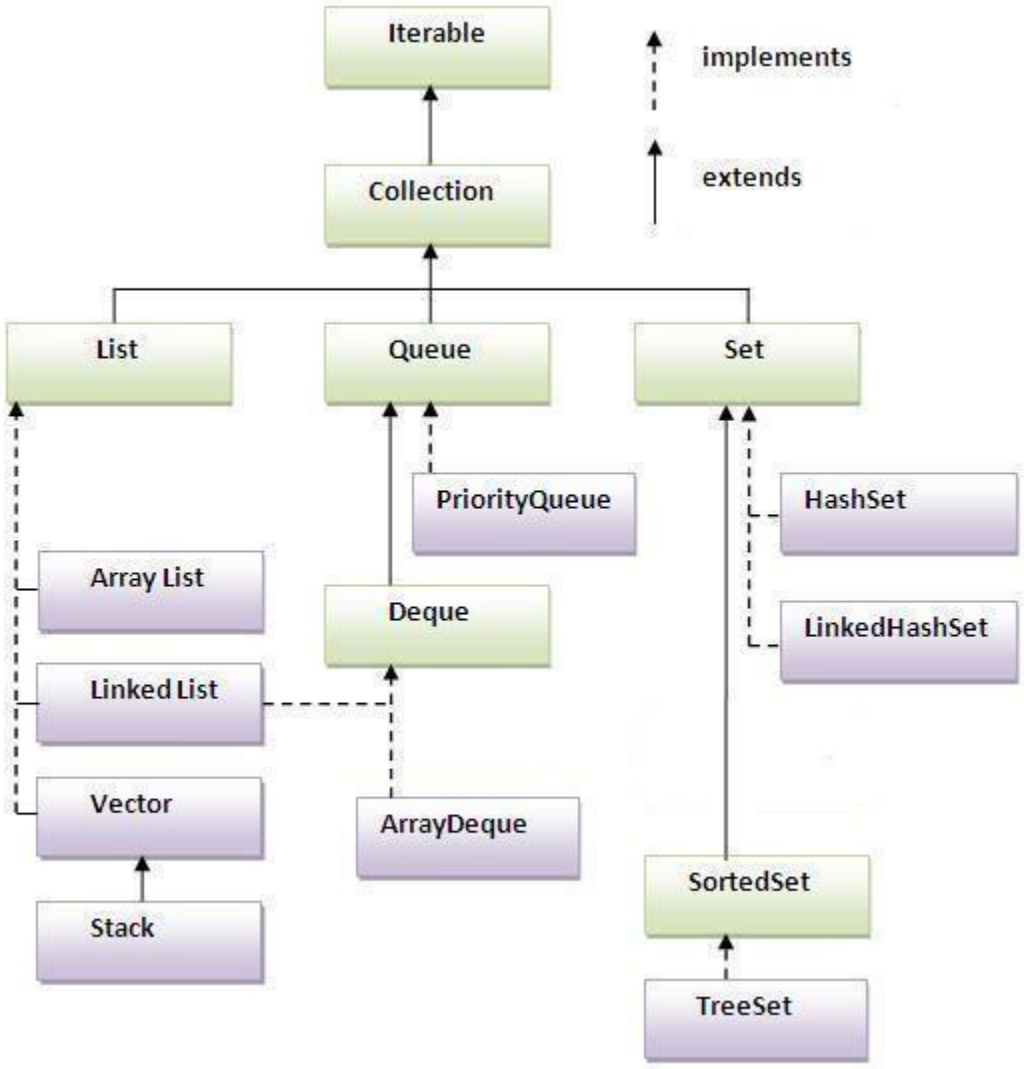
What is Collection framework

Collection framework represents a unified architecture for storing and manipulating group of objects. It has:

1. Interfaces and its implementations i.e. classes
2. Algorithm

Hierarchy of Collection Framework

Let us see the hierarchy of collection framework.The **java.util** package contains all the classes and interfaces for Collection framework.



Methods of Collection interface

There are many methods declared in the Collection interface. They are as follows:

No.	Method	Description
1	public boolean add(Object element)	is used to insert an element in this collection.
2	public boolean addAll(collection c)	is used to insert the specified collection elements in the invoking collection.
3	public boolean remove(Object element)	is used to delete an element from this collection.

4	public boolean the removeAll(Collection c)	is used to delete all the elements of specified collection from invoking collection.
5	public boolean retainAll(Collection c)	is used to delete all the elements of invoking collection except the specified collection.
6	public int size()	return the total number of elements in the collection.
7	public void clear()	removes the total no of element from the collection.
8	public boolean contains(object element. element)	is used to search an
9	public boolean collection. containsAll(Collection c)	is used to search the specified collection in this
10	public Iterator iterator()	returns an iterator.
11	public Object[] toArray()	converts collection into array.
12	public boolean isEmpty()	checks if collection is empty.
13	public boolean equals(Object element)	matches two collection.
14	public int hashCode()	returns the hashcode number for collection.

Iterator interface

Iterator interface provides the facility of iterating the elements in forward direction only.

Methods of Iterator interface

There are only three methods in the Iterator interface. They are:

1. **public boolean hasNext()** it returns true if iterator has more elements.
2. **public object next()** it returns the element and moves the cursor pointer to the next element.
3. **public void remove()** it removes the last elements returned by the iterator. It is rarely used.

Java ArrayList class

- Java ArrayList class uses a dynamic array for storing the elements.It extends AbstractList class and implements List interface.
- Java ArrayList class can contain duplicate elements.
- Java ArrayList class maintains insertion order.
- Java ArrayList class is non synchronized.
- Java ArrayList allows random access because array works at the index basis.
- In Java ArrayList class, manipulation is slow because a lot of shifting needs to be occurred if any element is removed from the array list.

Java Non-generic Vs Generic Collection

Java collection framework was non-generic before JDK 1.5. Since 1.5, it is generic.

Java new generic collection allows you to have only one type of object in collection. Now it is type safe so typecasting is not required at run time.

Let's see the old non-generic example of creating java collection.

```
1. ArrayList al=new ArrayList();//creating old non-generic arraylist
```

Let's see the new generic example of creating java collection.

```
1. ArrayList<String> al=new ArrayList<String>();//creating new generic arraylist
```

In generic collection, we specify the type in angular braces. Now ArrayList is forced to have only specified type of objects in it. If you try to add another type of object, it gives *compile time error*.

Example of Java ArrayList class

```
1.  import java.util.*;
2.  class TestCollection1{
3.   public static void main(String args[]){
4.
5.    ArrayList<String> al=new ArrayList<String>();//creating arraylist
6.    al.add("Ravi");//adding object in arraylist
7.    al.add("Vijay");
8.    al.add("Ravi");
9.    al.add("Ajay");
10.
11.   Iterator itr=al.iterator();//getting Iterator from arraylist to traverse elements
12.   while(itr.hasNext()){
13.    System.out.println(itr.next());
14.  }
15.  }
16. }
```

```
Ravi
Vijay
Ravi
Ajay
```

Two ways to iterate the elements of collection in java

1. By Iterator interface.
2. By for-each loop.

In the above example, we have seen traversing ArrayList by Iterator. Let's see the example to traverse ArrayList elements using for-each loop.

Iterating the elements of Collection by for-each loop

```
1. import java.util.*;
2. class TestCollection2{
3.  public static void main(String args[]){
4.   ArrayList<String> al=new ArrayList<String>();
5.   al.add("Ravi");
6.   al.add("Vijay");
7.   al.add("Ravi");
8.   al.add("Ajay");
9.   for(String obj:al)
10.    System.out.println(obj);
11.  }
12. }
```

```
Ravi
Vijay
Ravi
Ajay
```

User-defined class objects in Java ArrayList

```
1. class Student{
2.   int rollno;
3.   String name;
4.   int age;
5.   Student(int rollno,String name,int age){
6.    this.rollno=rollno;
7.    this.name=name;
8.   this.age=age;
9.   }
10. }
```

```
1. import java.util.*;
2. public class TestCollection3{
3.  public static void main(String args[]){
4.   //Creating user-defined class objects
5.   Student s1=new Student(101,"Sonoo",23);
6.   Student s2=new Student(102,"Ravi",21);
7.   Student s2=new Student(103,"Hanumat",25);
8.
9.   ArrayList<Student> al=new ArrayList<Student>();//creating arraylist
10.  al.add(s1);//adding Student class object
11.  al.add(s2);
12.  al.add(s3);


13.  Iterator itr=al.iterator();
14.  //traversing elements of ArrayList object
15.  while(itr.hasNext()){
16.   Student st=(Student)itr.next();
```

```
17.     System.out.println(st.rollno+" "+st.name+" "+st.age);
19.  }
20.  }
21. }
```

```
101 Sonoo 23
102 Ravi 21
103 Hanumat 25
```

Example of addAll(Collection c) method

```
1.  import java.util.*;
2.  class TestCollection4{
3.   public static void main(String args[]){
4.
5.    ArrayList<String> al=new ArrayList<String>();
6.    al.add("Ravi");
7.    al.add("Vijay");
8.    al.add("Ajay");
10.   ArrayList<String> al2=new ArrayList<String>();
11.   al2.add("Sonoo");
12.   al2.add("Hanumat")
; 13.
14. al.addAll(al2);
15.
16.   Iterator itr=al.iterator();
17.   while(itr.hasNext()){
18.    System.out.println(itr.next());
19.  }
20.  }
21. }
```

```
Ravi
Vijay
Ajay
Sonoo
Hanumat
```

Example of removeAll() method

```
1.  import java.util.*;
2.  class TestCollection5{
3.   public static void main(String args[]){


5.    ArrayList<String> al=new ArrayList<String>();
6.    al.add("Ravi");
7.    al.add("Vijay");
8.    al.add("Ajay");
9.
```

```
10.   ArrayList<String> al2=new ArrayList<String>();
11.   al2.add("Ravi");
12.   al2.add("Hanumat")
; 13.
14. al.removeAll(al2);
15.
16.   System.out.println("iterating the elements after removing the elements of al2...");
17.   Iterator itr=al.iterator();
18.   while(itr.hasNext()){
19.    System.out.println(itr.next());
20. }
21.
22.  }
23. }
```

```
iterating the elements after removing the elements of al2...
Vijay
Ajay
```

Example of retainAll() method

```
1.   import java.util.*;
2.   class TestCollection6{
3.    public static void main(String args[]){
4.     ArrayList<String> al=new ArrayList<String>();
5.     al.add("Ravi");
6.     al.add("Vijay");
7.     al.add("Ajay");
8.     ArrayList<String> al2=new ArrayList<String>();
9.     al2.add("Ravi");
10.   al2.add("Hanumat")
; 11.
12. al.retainAll(al2);
13.
14.   System.out.println("iterating the elements after retaining the elements of al2...");
15.   Iterator itr=al.iterator();
16.   while(itr.hasNext()){
17.    System.out.println(itr.next());
18.  }
19.  }
20. }
```

```
iterating the elements after retaining the elements of al2...
Ravi
```

Byte Streams

Java byte streams are used to perform input and output of 8-bit bytes. Though there are many classes related to byte streams but the most frequently used classes are , **FileInputStream** and **FileOutputStream**. Following is an example which makes use of these two classes to copy an input file into an output file:

```
import  java.io.*;

public class CopyFile {
  public static void main(String args[]) throws IOException
  {
    FileInputStream in = null;
    FileOutputStream out = null;

    try {
      in = new FileInputStream("input.txt"); out
      = new FileOutputStream("output.txt");
      int c;


      while ((c = in.read()) != -1)
        { out.write(c);
      }
    }finally {
      if (in != null) {
        in.close();
      }
      if (out != null) {
        out.close();
      }
    }
  }
}
```

Now let's have a file **input.txt** with the following content:

```
This is test for copy file.
```

As a next step, compile above program and execute it, which will result in creating output.txt file with the same content as we have in input.txt. So let's put above code in CopyFile.java file and do the following:

```
$javac CopyFile.java
$java CopyFile
```

Character Streams

Java **Byte** streams are used to perform input and output of 8-bit bytes, where as Java **Character** streams are used to perform input and output for 16-bit unicode. Though there are many classes related to character streams but the most frequently used classes are , **FileReader** and **FileWriter.**. Though internally FileReader uses FileInputStream and FileWriter uses FileOutputStream but here major difference is that FileReader reads two bytes at a time and FileWriter writes two bytes at a time.

We can re-write above example which makes use of these two classes to copy an input file (having unicode characters) into an output file:

```
import  java.io.*;

public class CopyFile {
  public static void main(String args[]) throws IOException
  {
    FileReader in = null;
    FileWriter out = null;

    try {
      in = new FileReader("input.txt");
      out = new FileWriter("output.txt");

      int c;
      while ((c = in.read()) != -1)
        { out.write(c);
      }


    }finally {
      if (in != null) {
        in.close();
      }
      if (out != null) {
        out.close();
      }
    }
  }
}
```

Now let's have a file **input.txt** with the following content:

```
This is test for copy file.
```

As a next step, compile above program and execute it, which will result in creating output.txt file with the same content as we have in input.txt. So let's put above code in CopyFile.java file and do the following:

```
$javac CopyFile.java
$java CopyFile
```

Standard Streams

All the programming languages provide support for standard I/O where user's program can take input from a keyboard and then produce output on the computer screen. If you are aware if C or C++ programming languages, then you must be aware of three standard devices STDIN, STDOUT and STDERR. Similar way Java provides following three standard streams

- **Standard Input:** This is used to feed the data to user's program and usually a keyboard is used as standard input stream and represented as **System.in**.

- **Standard Output:** This is used to output the data produced by the user's program and usually a computer screen is used to standard output stream and represented as **System.out**.
- **Standard Error:** This is used to output the error data produced by the user's program and usually a computer screen is used to standard error stream and represented as **System.err**.

Following is a simple program which creates **InputStreamReader** to read standard input stream until the user types a "q":

```
import java.io.*;

public class ReadConsole {
   public static void main(String args[]) throws IOException
   {
      InputStreamReader cin = null;
      try {
         cin = new InputStreamReader(System.in);
         System.out.println("Enter characters, 'q' to quit.");
         char c;
         do {

            c = (char) cin.read();
            System.out.print(c);
         } while(c !=
      'q'); }finally {
         if (cin != null)
            { cin.close();
         }
      }
   }
}
```

Let's keep above code in ReadConsole.java file and try to compile and execute it as below. This program continues reading and outputting same character until we press 'q':

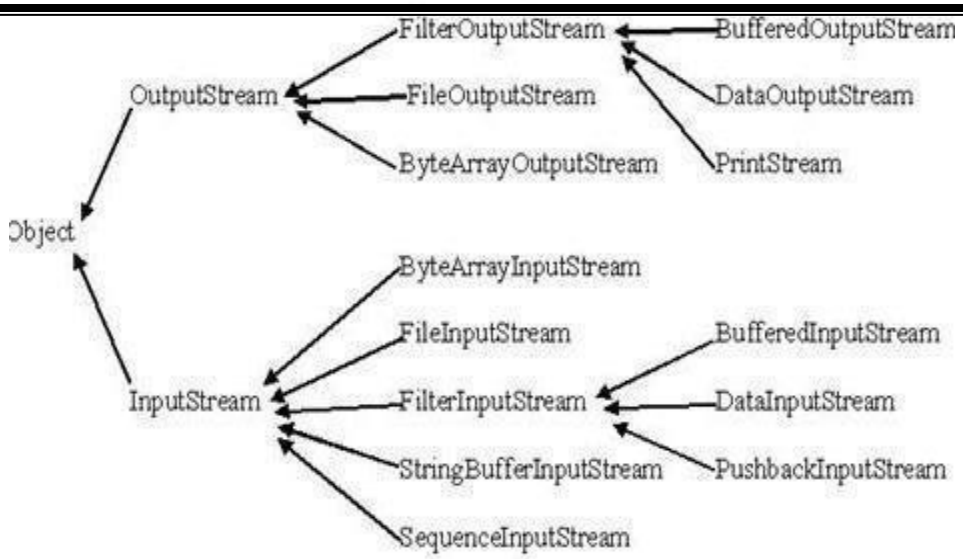
```
$javac ReadConsole.java
$java ReadConsole
Enter characters, 'q' to quit.
1 1

e
e
q
q
```

Reading and Writing Files:

As described earlier, A stream can be defined as a sequence of data. The **InputStream** is used to read data from a source and the **OutputStream** is used for writing data to a destination.

Here is a hierarchy of classes to deal with Input and Output streams.



The two important streams are **FileInputStream** and **FileOutputStream**, which would be discussed in this tutorial:

FileInputStream:

This stream is used for reading data from the files. Objects can be created using the keyword new and there are several types of constructors available.

Following constructor takes a file name as a string to create an input stream object to read the file.:

```
InputStream f = new FileInputStream("C:/java/hello");
```

Following constructor takes a file object to create an input stream object to read the file. First we create a file object using File() method as follows:

```
File f = new File("C:/java/hello");
InputStream f = new FileInputStream(f);
```

Once you have *InputStream* object in hand, then there is a list of helper methods which can be used to read to stream or to do other operations on the stream.

SN Methods with Description

public void close() throws IOException{}

1 This method closes the file output stream. Releases any system resources associated with the file. Throws an IOException.

protected void finalize()throws IOException {}

2 This method cleans up the connection to the file. Ensures that the close method of this file output stream is called when there are no more references to this stream. Throws an IOException.

public int read(int r)throws IOException{}

3 This method reads the specified byte of data from the InputStream. Returns an int. Returns the next byte of data and -1 will be returned if it's end of file.

public int read(byte[] r) throws IOException{}

4 This method reads r.length bytes from the input stream into an array. Returns the total number of bytes read. If end of file -1 will be returned.

5 public int available() throws IOException{}

Gives the number of bytes that can be read from this file input stream. Returns an int.

There are other important input streams available, for more detail you can refer to the following links:

- ByteArrayInputStream
- DataInputStream

FileOutputStream:

FileOutputStream is used to create a file and write data into it. The stream would create a file, if it doesn't already exist, before opening it for output.

Here are two constructors which can be used to create a FileOutputStream object.

Following constructor takes a file name as a string to create an input stream object to write the file:

OutputStream f = new FileOutputStream("C:/java/hello")

Following constructor takes a file object to create an output stream object to write the file. First, we create a file object using File() method as follows:

File f = new File("C:/java/hello");
OutputStream f = new FileOutputStream(f);

Once you have *OutputStream* object in hand, then there is a list of helper methods, which can be used to write to stream or to do other operations on the stream.

SN	Methods with Description
	public void close() throws IOException{}
1	This method closes the file output stream. Releases any system resources associated with the file. Throws an IOException.
	protected void finalize()throws IOException {}
2	This method cleans up the connection to the file. Ensures that the close method of this file output stream is called when there are no more references to this stream. Throws an IOException.
3	**public void write(int w)throws IOException{}**
	This methods writes the specified byte to the output stream.
4	**public void write(byte[] w)**
	Writes w.length bytes from the mentioned byte array to the OutputStream.

There are other important output streams available, for more detail you can refer to the following links:

- ByteArrayOutputStream
- DataOutputStream

Example:

Following is the example to demonstrate InputStream and OutputStream:

```
import java.io.*;

public class fileStreamTest{

  public static void main(String args[]){

  try{
    byte bWrite [] = {11,21,3,40,5};
    OutputStream os = new FileOutputStream("test.txt");
    for(int x=0; x < bWrite.length ; x++){
      os.write( bWrite[x] ); // writes the bytes
    }
    os.close();


    InputStream is = new FileInputStream("test.txt");

    int size = is.available();


    for(int i=0; i< size; i++){
      System.out.print((char)is.read() + " ");
    }
    is.close();
  }catch(IOException e){
    System.out.print("Exception");
  }
  }
}
```

The above code would create file test.txt and would write given numbers in binary format. Same would be output on the stdout screen.

File Navigation and I/O:

There are several other classes that we would be going through to get to know the basics of File Navigation and I/O.

- File Class
- FileReader Class
- FileWriter Class

Directories in Java:

A directory is a File which can contains a list of other files and directories. You use **File** object to create directories, to list down files available in a directory. For complete detail check a list of all the methods which you can call on File object and what are related to directories.

Creating Directories:

There are two useful **File** utility methods, which can be used to create directories:

- The **mkdir( )** method creates a directory, returning true on success and false on failure. Failure indicates that the path specified in the File object already exists, or that the directory cannot be created because the entire path does not exist yet.
- The **mkdirs()** method creates both a directory and all the parents of the directory.

Following example creates "/tmp/user/java/bin" directory:

```
import java.io.File;

public class CreateDir {
  public static void main(String args[]) {
    String dirname = "/tmp/user/java/bin";
    File d = new File(dirname);
    // Create directory now.
    d.mkdirs();
  }
}
```

Compile and execute above code to create "/tmp/user/java/bin".

Note: Java automatically takes care of path separators on UNIX and Windows as per conventions. If you use a forward slash (/) on a Windows version of Java, the path will still resolve correctly.

Listing Directories:

You can use **list( )** method provided by **File** object to list down all the files and directories available in a directory as follows:

```
import java.io.File;

public class ReadDir {
  public static void main(String[] args) {

    File file = null;
    String[] paths;

    try{
      // create new file object
      file = new File("/tmp");

      // array of files and directory
      paths = file.list();

      // for each name in the path array
      for(String path:paths)
      {
        // prints filename and directory name
        System.out.println(path);
      }
    }catch(Exception e){
      // if any error occurs
      e.printStackTrace();
    }
  }
}
```

This would produce following result based on the directories and files available in your **/tmp** directory:

```
test1.txt
test2.txt
ReadDir.java
ReadDir.class
```

JDBC

JDBC stands for **J**ava **D**ata**b**ase **C**onnectivity, which is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases.

The JDBC library includes APIs for each of the tasks commonly associated with database usage:

- Making a connection to a database
- Creating SQL or MySQL statements
- Executing that SQL or MySQL queries in the database
- Viewing & Modifying the resulting records

Fundamentally, JDBC is a specification that provides a complete set of interfaces that allows for portable access to an underlying database. Java can be used to write different types of executables, such as:

- Java Applications
- Java Applets
- Java Servlets
- Java ServerPages (JSPs)
- Enterprise JavaBeans (EJBs)

All of these different executables are able to use a JDBC driver to access a database and take advantage of the stored data.

JDBC provides the same capabilities as ODBC, allowing Java programs to contain database-independent code.

JDBC Architecture:

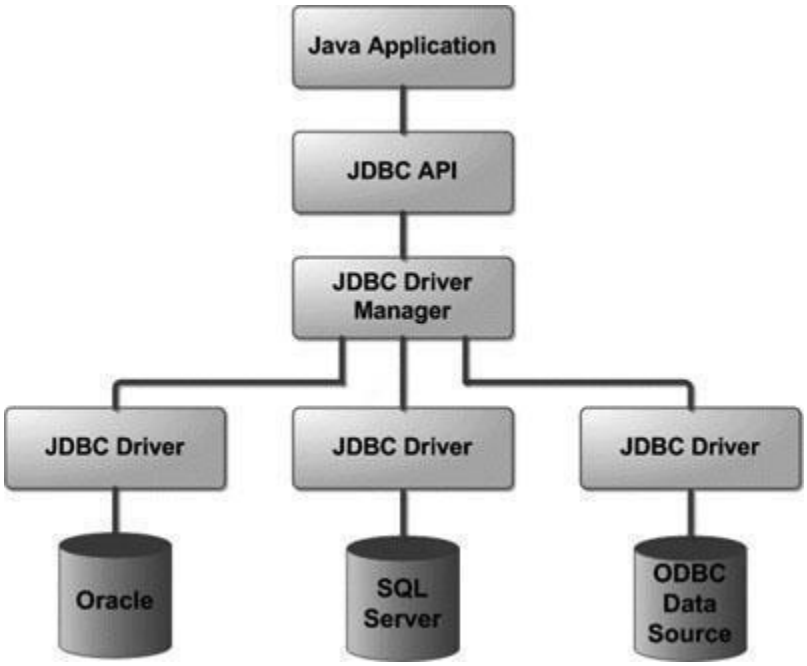
The JDBC API supports both two-tier and three-tier processing models for database access but in general JDBC Architecture consists of two layers:

- **JDBC API:** This provides the application-to-JDBC Manager connection.
- **JDBC Driver API:** This supports the JDBC Manager-to-Driver Connection.

The JDBC API uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.

The JDBC driver manager ensures that the correct driver is used to access each data source. The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.

Following is the architectural diagram, which shows the location of the driver manager with respect to the JDBC drivers and the Java application:



Common JDBC Components:

The JDBC API provides the following interfaces and classes:

- **DriverManager:** This class manages a list of database drivers. Matches connection requests from the java application with the proper database driver using communication subprotocol. The first driver that recognizes a certain subprotocol under JDBC will be used to establish a database Connection.
- **Driver:** This interface handles the communications with the database server. You will interact directly with Driver objects very rarely. Instead, you use DriverManager objects, which manages objects of this type. It also abstracts the details associated with working with Driver objects
- **Connection :** This interface with all methods for contacting a database. The connection object represents communication context, i.e., all communication with database is through connection object only.
- **Statement :** You use objects created from this interface to submit the SQL statements to the database. Some derived interfaces accept parameters in addition to executing stored procedures.
- **ResultSet:** These objects hold data retrieved from a database after you execute an SQL query using Statement objects. It acts as an iterator to allow you to move through its data.
- **SQLException:** This class handles any errors that occur in a database application.

Creating JDBC Application:

There are following six steps involved in building a JDBC application:

- **Import the packages .** Requires that you include the packages containing the JDBC classes needed for database programming. Most often, using *import java.sql.** will suffice.
- **Register the JDBC driver .** Requires that you initialize a driver so you can open a communications channel with the database.

- **Open a connection .** Requires using the *DriverManager.getConnection()* method to create a Connection object, which represents a physical connection with the database.
- **Execute a query .** Requires using an object of type Statement for building and submitting an SQL statement to the database.
- **Extract data from result set .** Requires that you use the appropriate *ResultSet.getXXX()* method to retrieve the data from the result set.
- **Clean up the environment .** Requires explicitly closing all database resources versus relying on the JVM's garbage collection.

Creating JDBC Application:

- There are six steps involved in building a JDBC application which I'm going to brief in this tutorial:
- **Import the packages:**
- This requires that you include the packages containing the JDBC classes needed for database programming. Most often, using import java.sql.* will suffice as follows:
- //STEP 1. Import required packages
- import java.sql.*;
- **Register the JDBC driver:**
- This requires that you initialize a driver so you can open a communications channel with the database. Following is the code snippet to achieve this:
- //STEP 2: Register JDBC driver
- Class.forName("com.mysql.jdbc.Driver");
- **Open a connection:**
- This requires using the DriverManager.getConnection() method to create a Connection object, which represents a physical connection with the database as follows:
- //STEP 3: Open a connection
- // Database credentials
- static final String USER = "username";
- static final String PASS = "password";
- System.out.println("Connecting to database...");
- conn = DriverManager.getConnection(DB_URL,USER,PASS);
- **Execute a query:**
- This requires using an object of type Statement or PreparedStatement for building and submitting an SQL statement to the database as follows:
- //STEP 4: Execute a query
- System.out.println("Creating statement...");
- stmt = conn.createStatement();
- String sql;
- sql = "SELECT id, first, last, age FROM Employees";
- ResultSet rs = stmt.executeQuery(sql);
- If there is an SQL UPDATE,INSERT or DELETE statement required, then following code snippet would be required:
- //STEP 4: Execute a query
- System.out.println("Creating statement...");
- stmt = conn.createStatement();
- String sql;
- sql = "DELETE FROM Employees";
- ResultSet rs = stmt.executeUpdate(sql);
- **Extract data from result set:**
- This step is required in case you are fetching data from the database. You can use the appropriate ResultSet.getXXX() method to retrieve the data from the result set as follows:

```
//STEP 5: Extract data from result set
while(rs.next()){
   //Retrieve by column name
   int id = rs.getInt("id");
   int age = rs.getInt("age");
   String first = rs.getString("first");
   String last = rs.getString("last");

   //Display values
   System.out.print("ID: " + id);
   System.out.print(", Age: " + age);
   System.out.print(", First: " + first);
   System.out.println(", Last: " + last);
}
```

Clean up the environment:

You should explicitly close all database resources versus relying on the JVM's garbage collection as follows:

```
//STEP 6: Clean-up environment
rs.close();
stmt.close();
conn.close();
```

First JDBC Program:

Based on the above steps, we can have following consolidated sample code which we can use as a template while writing our JDBC code:

This sample code has been written based on the environment and database setup done in Environment chapter.

```
//STEP 1. Import required packages
import java.sql.*;

public class FirstExample {
   // JDBC driver name and database URL
   static final String JDBC_DRIVER = "com.mysql.jdbc.Driver";
   static final String DB_URL = "jdbc:mysql://localhost/EMP";

   // Database credentials
   static final String USER = "username";

   static final String PASS = "password";

   public static void main(String[] args) {
   Connection conn = null;
   Statement stmt = null;
   try{
      //STEP 2: Register JDBC driver
      Class.forName("com.mysql.jdbc.Driver");

      //STEP 3: Open a connection
      System.out.println("Connecting to database...");
      conn = DriverManager.getConnection(DB_URL,USER,PASS);

      //STEP 4: Execute a query
      System.out.println("Creating statement...");
```

```
            stmt = conn.createStatement();
            String sql;
            sql = "SELECT id, first, last, age FROM Employees";
            ResultSet rs = stmt.executeQuery(sql);

            //STEP 5: Extract data from result set
            while(rs.next()){
              //Retrieve by column name
              int id = rs.getInt("id");
              int age = rs.getInt("age");
              String first = rs.getString("first");
              String last = rs.getString("last");

              //Display values
              System.out.print("ID: " + id);
              System.out.print(", Age: " + age);
              System.out.print(", First: " + first);
              System.out.println(", Last: " + last);
            }
            //STEP 6: Clean-up environment
            rs.close();
            stmt.close();
            conn.close();
          }catch(SQLException se){
            //Handle errors for JDBC
            se.printStackTrace();
          }catch(Exception e){
            //Handle errors for Class.forName
            e.printStackTrace();
        }finally{
            //finally block used to close resources
            try{
              if(stmt!=null)
                 stmt.close();
            }catch(SQLException se2){
            }// nothing we can do
            try{
              if(conn!=null)
                 conn.close();
            }catch(SQLException se){
              se.printStackTrace();
            }//end finally try
          }//end try
          System.out.println("Goodbye!");
        }//end main
        }//end FirstExample
```

Now let us compile above example as follows:

```
C:\>javac FirstExample.java
C:\>
```

When you run **FirstExample**, it produces following result:

```
C:\>java FirstExample
```

```
Connecting to database...
Creating statement...
ID: 100, Age: 18, First: Zara, Last: Ali
ID: 101, Age: 25, First: Mahnaz, Last: Fatma
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
C:\>
```

SQLException Methods:

- A SQLException can occur both in the driver and the database. When such an exception occurs, an object of type SQLException will be passed to the catch clause.
- The passed SQLException object has the following methods available for retrieving additional information about the exception:

Method	Description
getErrorCode( )	Gets the error number associated with the exception.
getMessage( )	Gets the JDBC driver's error message for an error handled by the driver or gets the Oracle error number and message for a database error.
getSQLState( )	Gets the XOPEN SQLstate string. For a JDBC driver error, no useful information is returned from this method. For a database error, the five-digit XOPEN SQLstate code is returned. This method can return null.
getNextException( )	Gets the next Exception object in the exception chain.
printStackTrace( )	Prints the current exception, or throwable, and its backtrace to a standard error stream.
printStackTrace(PrintStream s)	Prints this throwable and its backtrace to the print stream you specify.
printStackTrace(PrintWriter w)	Prints this throwable and its backtrace to the print writer you specify.

- By utilizing the information available from the Exception object, you can catch an exception and continue your program appropriately. Here is the general form of a try block:
- try {
- // Your risky code goes between these curly braces!!!
- }
- catch(Exception ex) {
- // Your exception handling code goes between these
- // curly braces, similar to the exception clause
- // in a PL/SQL block.
- }
- finally {
- // Your must-always-be-executed code goes between these
- // curly braces. Like closing database connection.
- }
- **JDBC - Data Types:**

- The following table summarizes the default JDBC data type that the Java data type is converted to when you call the setXXX() method of the PreparedStatement or CallableStatement object or the ResultSet.updateXXX() method.

SQL	JDBC/Java	setXXX	updateXXX
VARCHAR	java.lang.String	setString	updateString
CHAR	java.lang.String	setString	updateString
LONGVARCHAR	java.lang.String	setString	updateString
BIT	Boolean	setBoolean	updateBoolean
NUMERIC	java.math.BigDecimal	setBigDecimal	updateBigDecimal
TINYINT	Byte	setByte	updateByte
SMALLINT	Short	setShort	updateShort
INTEGER	Int	setInt	updateInt
BIGINT	Long	setLong	updateLong
REAL	Float	setFloat	updateFloat
FLOAT	Float	setFloat	updateFloat
DOUBLE	Double	setDouble	updateDouble
VARBINARY	byte[ ]	setBytes	updateBytes
BINARY	byte[ ]	setBytes	updateBytes
DATE	java.sql.Date	setDate	updateDate
TIME	java.sql.Time	setTime	updateTime
TIMESTAMP	java.sql.Timestamp	setTimestamp	updateTimestamp
CLOB	java.sql.Clob	setClob	updateClob
BLOB	java.sql.Blob	setBlob	updateBlob
ARRAY	java.sql.Array	setARRAY	updateARRAY
REF	java.sql.Ref	SetRef	updateRef
STRUCT	java.sql.Struct	SetStruct	updateStruct

- JDBC 3.0 has enhanced support for BLOB, CLOB, ARRAY, and REF data types. The ResultSet object now has updateBLOB(), updateCLOB(), updateArray(), and updateRef() methods that enable you to directly manipulate the respective data on the server.
- The setXXX() and updateXXX() methods enable you to convert specific Java types to specific JDBC data types. The methods, setObject() and updateObject(), enable you to map almost any Java type to a JDBC data type.
- ResultSet object provides corresponding getXXX() method for each data type to retrieve column value. Each method can be used with column name or by its ordinal position.

SQL	**JDBC/Java**	**setXXX**	**getXXX**
VARCHAR	java.lang.String	setString	getString
CHAR	java.lang.String	setString	getString
LONGVARCHAR	java.lang.String	setString	getString
BIT	Boolean	setBoolean	getBoolean
NUMERIC	java.math.BigDecimal	setBigDecimal	getBigDecimal
TINYINT	Byte	setByte	getByte
SMALLINT	Short	setShort	getShort
INTEGER	Int	setInt	getInt
BIGINT	Long	setLong	getLong
REAL	Float	setFloat	getFloat
FLOAT	Float	setFloat	getFloat
DOUBLE	Double	setDouble	getDouble
VARBINARY	byte[ ]	setBytes	getBytes
BINARY	byte[ ]	setBytes	getBytes
DATE	java.sql.Date	setDate	getDate
TIME	java.sql.Time	setTime	getTime
TIMESTAMP	java.sql.Timestamp	setTimestamp	getTimestamp
CLOB	java.sql.Clob	setClob	getClob
BLOB	java.sql.Blob	setBlob	getBlob
ARRAY	java.sql.Array	setARRAY	getARRAY
REF	java.sql.Ref	SetRef	getRef
STRUCT	java.sql.Struct	SetStruct	getStruct

Sample Code:

This sample example can serve as a **template** when you need to create your own JDBC application in the future.

This sample code has been written based on the environment and database setup done in previous chapter.

Copy and past following example in FirstExample.java, compile and run as follows:

```
//STEP 1. Import required
packages import java.sql.*;

public class FirstExample {
  // JDBC driver name and database URL
```

```
static final String JDBC_DRIVER = "com.mysql.jdbc.Driver";
static final String DB_URL = "jdbc:mysql://localhost/EMP";

// Database credentials
static final String USER = "username";
static final String PASS = "password";

public static void main(String[] args) {
Connection conn = null;
Statement stmt = null;
try{
   //STEP 2: Register JDBC driver
   Class.forName("com.mysql.jdbc.Driver");

   //STEP 3: Open a connection
   System.out.println("Connecting to database...");
   conn = DriverManager.getConnection(DB_URL,USER,PASS);

   //STEP 4: Execute a query
   System.out.println("Creating statement...");
   stmt = conn.createStatement();
   String sql;
   sql = "SELECT id, first, last, age FROM Employees";
   ResultSet rs = stmt.executeQuery(sql);
   //STEP 5: Extract data from result set
   while(rs.next()){
      //Retrieve by column name
      int id = rs.getInt("id");
      int age = rs.getInt("age");
      String first = rs.getString("first");

      String last = rs.getString("last");

      //Display values
      System.out.print("ID: " + id);
      System.out.print(", Age: " + age);
      System.out.print(", First: " + first);
      System.out.println(", Last: " + last);
   }
   //STEP 6: Clean-up environment
   rs.close();
   stmt.close();
   conn.close();
}catch(SQLException se){
   //Handle errors for JDBC
   se.printStackTrace();
}catch(Exception e){
   //Handle errors for Class.forName
   e.printStackTrace();
}finally{
   //finally block used to close resources
   try{
```

```
      if(stmt!=null)
         stmt.close();
    }catch(SQLException
    se2){ }// nothing we can do
    try{
      if(conn!=null)
         conn.close();
    }catch(SQLException se){
      se.printStackTrace();
    }//end finally try
  }//end try
  System.out.println("Goodbye!");
}//end main
}//end FirstExample
```

Now let us compile above example as follows:

```
C:\>javac FirstExample.java
C:\>
```

When you run **FirstExample**, it produces following result:

```
C:\>java FirstExample
Connecting to database...
Creating statement...
ID: 100, Age: 18, First: Zara, Last: Ali
ID: 101, Age: 25, First: Mahnaz, Last: Fatma
ID: 102, Age: 30, First: Zaid, Last: Khan
ID: 103, Age: 28, First: Sumit, Last: Mittal
C:\>
```

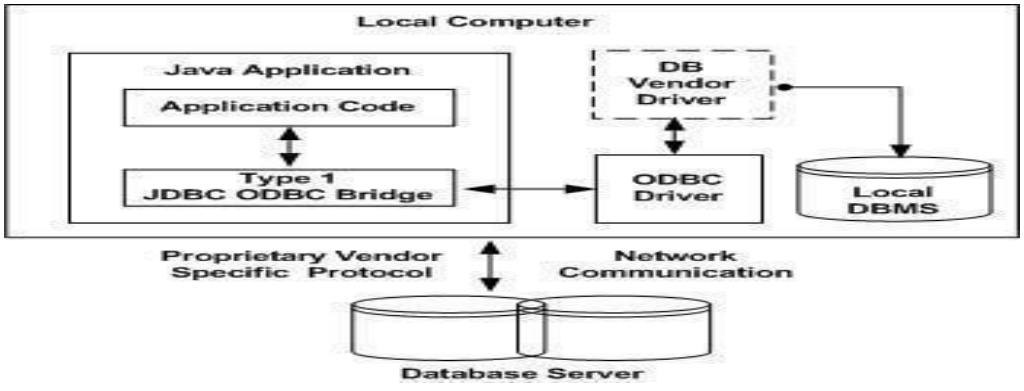
JDBC Drivers Types:

JDBC driver implementations vary because of the wide variety of operating systems and hardware platforms in which Java operates. Sun has divided the implementation types into four categories, Types 1, 2, 3, and 4, which is explained below:

Type 1: JDBC-ODBC Bridge Driver:

In a Type 1 driver, a JDBC bridge is used to access ODBC drivers installed on each client machine. Using ODBC requires configuring on your system a Data Source Name (DSN) that represents the target database.

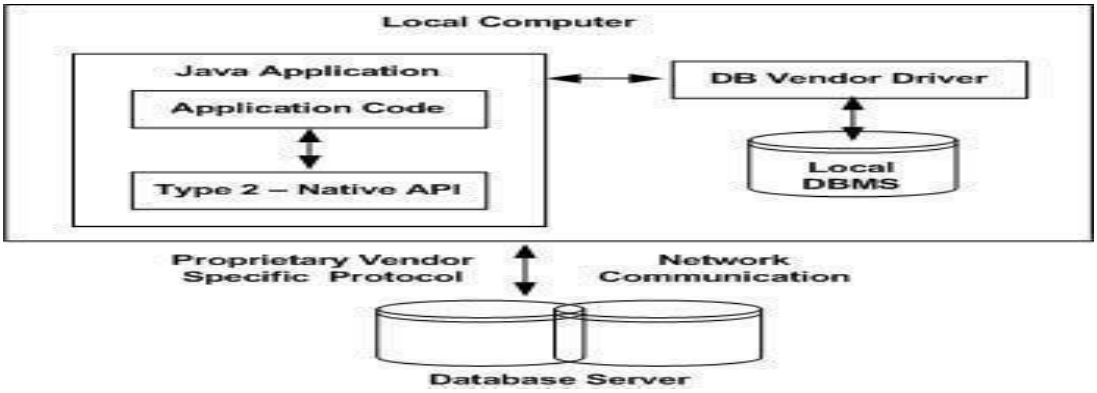
When Java first came out, this was a useful driver because most databases only supported ODBC access but now this type of driver is recommended only for experimental use or when no other alternative is available.



The JDBC-ODBC bridge that comes with JDK 1.2 is a good example of this kind of driver.

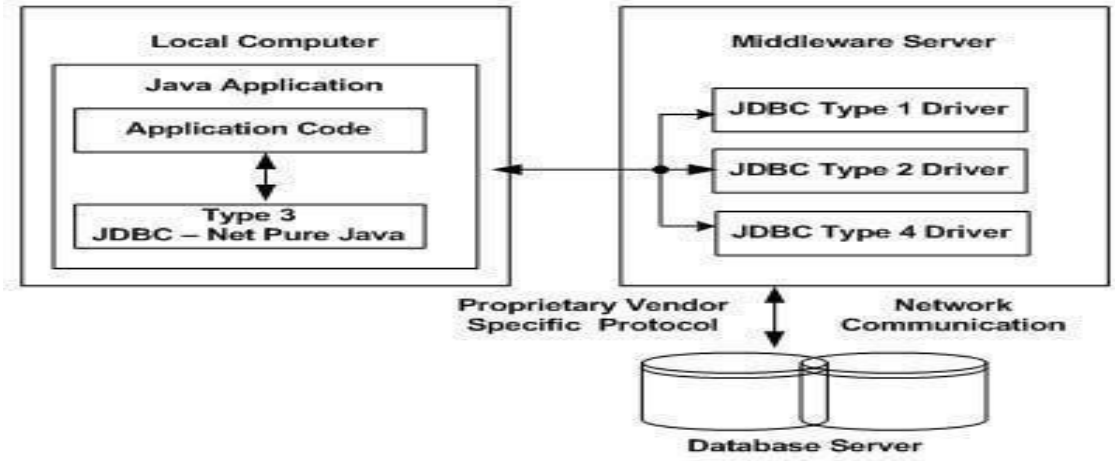
Type 2: JDBC-Native API:

In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls which are unique to the database. These drivers typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge, the vendor-specific driver must be installed on each client machine.
If we change the Database we have to change the native API as it is specific to a database and they are mostly obsolete now but you may realize some speed increase with a Type 2 driver, because it eliminates ODBC's overhead.



The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

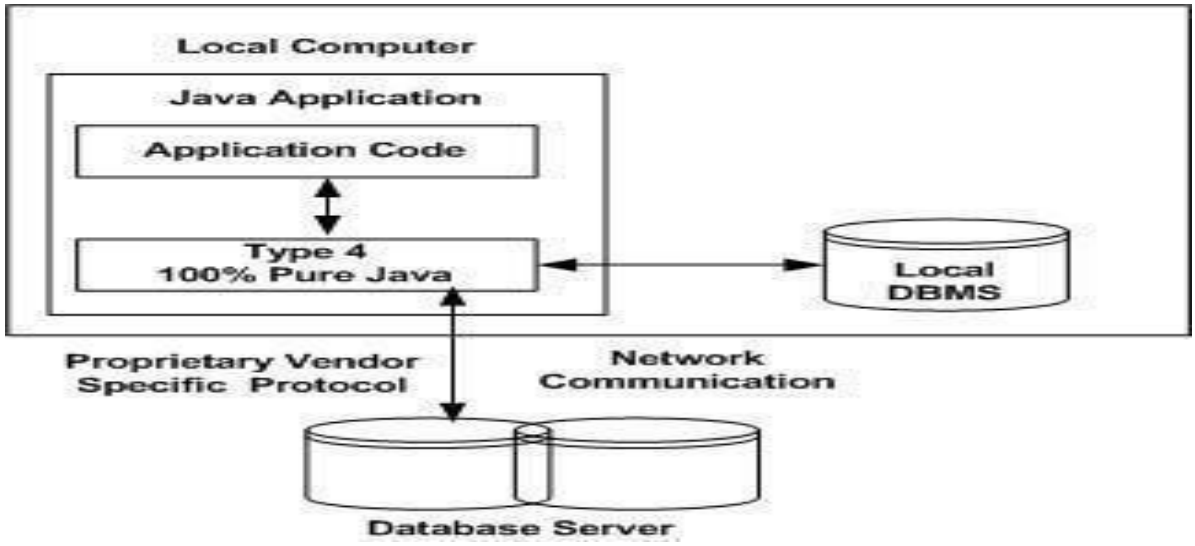
Type 3: JDBC-Net pure Java:
In a Type 3 driver, a three-tier approach is used to accessing databases. The JDBC clients use standard network sockets to communicate with an middleware application server. The socket information is then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server. This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases.



You can think of the application server as a JDBC "proxy," meaning that it makes calls for the client application. As a result, you need some knowledge of the application server's configuration in order to effectively use this driver type. Your application server might use a Type 1, 2, or 4 driver to communicate with the database, understanding the nuances will prove helpful.

Type 4: 100% pure Java:
In a Type 4 driver, a pure Java-based driver that communicates directly with vendor's database through socket connection. This is the highest performance driver available for the database and

is usually provided by the vendor itself.
This kind of driver is extremely flexible, you don't need to install special software on the client or server. Further, these drivers can be downloaded dynamically.



MySQL's Connector/J driver is a Type 4 driver. Because of the proprietary nature of their network protocols, database vendors usually supply type 4 drivers.

Which Driver should be used?

If you are accessing one type of database, such as Oracle, Sybase, or IBM, the preferred driver type is 4.

If your Java application is accessing multiple types of databases at the same time, type 3 is the preferred driver.

Type 2 drivers are useful in situations where a type 3 or type 4 driver is not available yet for your database.

The type 1 driver is not considered a deployment-level driver and is typically used for development and testing purposes only.

UNIT-V

GUI Programming with Swing –The AWT class hierarchy- introduction to swing- swing Vs AWT- overview of some swing components –JButton-JLabel- JTextField-JTextArea- simple applications- Layout management –Layout manager types –border- grid and flow

Event Handling: Events- Event sources- Event classes- Event Listeners- Delegation event model- Example: handling mouse events- Adapter classes.

INTRODUCTION OF SWING

The Swing-related classes are contained in **javax.swing** and its subpackages, such as **javax.swing.tree**. Many other Swing-related classes and interfaces exist that are not examined in this chapter.

The remainder of this chapter examines various Swing components and illustrates them through sample applets.

JApplet

Fundamental to Swing is the **JApplet** class, which extends **Applet**. Applets that useSwing must be subclasses of **JApplet**. **JApplet** is rich with functionality that is notfound in **Applet**. For example, **JApplet** supports various ―panes,‖heglass such pane, and the root pane. For the examples in this chapter, we will not be using most of **JApplet**'s enhanced features. However,**Applet**and**JApplet**oneis diffe important to this discussion, because it is used by the sample applets in this chapter. When adding a component to an instance of **JApplet**, do not invoke the **add( )** method of the applet. Instead, call **add( )** for the *content pane* of the **JApplet** object. The content pane can be obtained via the method shown here:

Container getContentPane( )

The **add( )** method of **Container** can be used to add a component to a content pane. Its form is shown here:

void add(*comp*)

Here, *comp* is the component to be added to the content pane.

Icons and Labels

In Swing, icons are encapsulated by the **ImageIcon** class, which paints an icon from an image. Two of its constructors are shown here:

ImageIcon(String *filename*)
ImageIcon(URL *url*)

The first form uses the image in the file named *filename*. The second form uses the image in the resource identified by *url*.

The **ImageIcon** class implements the **Icon** interface that declares the methods shown here:

Method Description

int getIconHeight( ) Returns the height of the icon in pixels.

int getIconWidth( ) Returns the width of the icon in pixels.

void paintIcon(Component *comp*, Graphics *g*,
int *x*, int *y*)

Paints the icon at position *x*, *y* on the
graphics context *g*. Additional
information about the paint
operation can be provided in *comp*.

Swing labels are instances of the **JLabel** class, which extends **JComponent**. It can display text and/or an icon. Some of its constructors are shown here:

JLabel(Icon *i*)
Label(String *s*)
JLabel(String *s*, Icon *i*, int *align*)

Here, *s* and *i* are the text and icon used for the label. The *align* argument is either **LEFT**,**RIGHT**, **CENTER**, **LEADING**, or **TRAILING**. These constants are defined in the **SwingConstants** interface, along with several others used by the Swing classes.

The icon and text associated with the label can be read and written by the following methods:

Icon getIcon( ) String
getText( ) void
setIcon(Icon *i*) void
setText(String *s*)
Here, *i* and *s* are the icon and text, respectively.

The following example illustrates how to create and display a label containing bothan icon and a string. The applet begins by getting its content pane. Next, an **ImageIcon** object is created for the file **france.gif**. This is used as the second argument to the **JLabel** constructor. The first and last arguments for the **JLabel** constructor are the label text and the alignment. Finally, the label is added to the content pane.

```
import java.awt.*;
import javax.swing.*;
/* <applet code="JLabelDemo" width=250 height=150> </applet>*/
public class JLabelDemo extends JApplet
{ public void init() {
// Get content pane
Container contentPane =
getContentPane(); // Create an icon
ImageIcon ii = new
ImageIcon("france.gif"); // Create a label
JLabel jl = new JLabel("France", ii,
JLabel.CENTER); // Add label to the content pane
contentPane.add(jl);
}
}
```

Output from this applet is shown here:

Text Fields

The Swing text field is encapsulated by the **JTextComponent** class, which extends **JComponent**. It provides functionality that is common to Swing text components. One of its subclasses is **JTextField**, which allows you to edit one line of text. Some of its constructors are shown here:

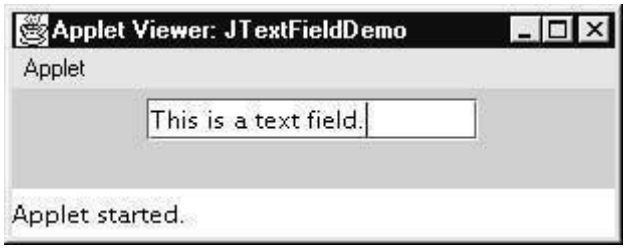
JTextField( )
JTextField(int *cols*)
JTextField(String *s*, int *cols*)
JTextField(String *s*)

Here, *s* is the string to be presented, and *cols* is the number of columns in the text field.

The following example illustrates how to create a text field. The applet begins by getting its content pane, and then a flow layout is assigned as its layout manager. Next, a **JTextField** object is created and is added to the content pane.

```
import java.awt.*;
import javax.swing.*;
/*
<applet code="JTextFieldDemo" width=300 height=50>
</applet>
*/
public class JTextFieldDemo extends JApplet {
JTextField jtf;
public void init() {
// Get content pane
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
// Add text field to content pane
jtf = new JTextField(15);
contentPane.add(jtf);
}
}
```

Output from this applet is shown here:



Buttons

Swing buttons provide features that are not found in the **Button** class defined by theAWT. For example, you can associate an icon with a Swing button. Swing buttons are subclasses of the **AbstractButton** class, which extends **JComponent**. **AbstractButton** contains many methods that allow you to control the behavior of buttons, check boxes, and radio buttons. For example, you can define different icons that are displayed for the component when it is disabled, pressed, or selected. Another icon can be used as a *rollover* icon, which is displayed when the mouse is positioned over that component.

The following are the methods that control this behavior:
void setDisabledIcon(Icon *di*)
void setPressedIcon(Icon *pi*)
void setSelectedIcon(Icon *si*)
void setRolloverIcon(Icon *ri*)

Here, *di*, *pi*, *si*, and *ri* are the icons to be used for these different conditions.
The text associated with a button can be read and written via the following methods:

String getText( ) void
setText(String *s*)

Here, *s* is the text to be associated with the button.

Concrete subclasses of **AbstractButton** generate action events when they are pressed. Listeners register and unregister for these events via the methods shown here:

void addActionListener(ActionListener *al*)
void removeActionListener(ActionListener *al*)
Here, *al* is the action listener.

AbstractButton is a superclass for push buttons, check boxes, and radio buttons.
Each is examined next.

The JButton Class

The **JButton** class provides the functionality of a push button. **JButton** allows an icon,a string, or both to be associated with the push button. Some of its constructors are shown here:
JButton(Icon *i*)

JButton(String *s*)
JButton(String *s*, Icon *i*)
Here, *s* and *i* are the string and icon used for the button.

Check Boxes

The **JCheckBox** class, which provides the functionality of a check box, is a concrete implementation of **AbstractButton**. Its immediate superclass is **JToggleButton**, which provides support for two-state buttons. Some of its constructors are shown here:

JCheckBox(Icon *i*)
JCheckBox(Icon *i*, boolean *state*)
JCheckBox(String *s*)
JCheckBox(String *s*, boolean *state*)
JCheckBox(String *s*, Icon *i*)
JCheckBox(String *s*, Icon *i*, boolean *state*)

Here, *i* is the icon for the button. The text is specified by *s*. If *state* is **true**, the check box is initially selected. Otherwise, it is not.

The state of the check box can be changed via the following
method: void setSelected(boolean *state*)
Here, *state* is **true** if the check box should be checked.

The following example illustrates how to create an applet that displays four checkboxes and a text field. When a check box is pressed, its text is displayed in the text field.

The content pane for the **JApplet** object is obtained, and a flow layout is assigned as itslayout manager. Next, four check boxes are added to the content pane, and icons are assigned for the normal, rollover, and selected states. The applet is then registered to receive item events. Finally, a text field is added to the content pane.

When a check box is selected or deselected, an item event is generated. This is handled by **itemStateChanged( )**. Inside **itemStateChanged( )**, the **getItem( )** method gets the **JCheckBox** object that generated the event. The **getText( )** method gets the text for that check box and uses it to set the text inside the text field.

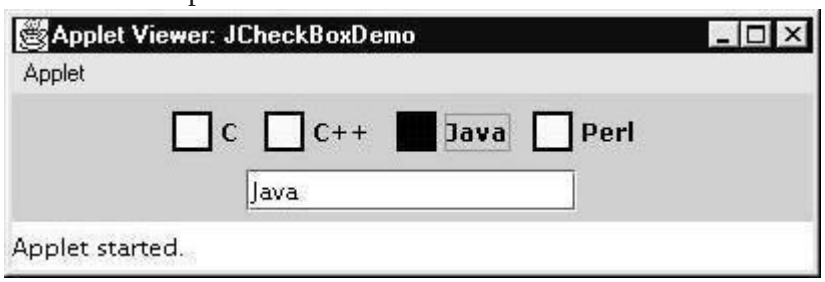
```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*; /*

<applet code="JCheckBoxDemo" width=400 height=50>
</applet>
*/
public class JCheckBoxDemo extends JApplet
implements ItemListener {
JTextField jtf;
public void init() {

// Get content pane
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
// Create icons
ImageIcon normal = new ImageIcon("normal.gif");
ImageIcon rollover = new ImageIcon("rollover.gif");
ImageIcon selected = new ImageIcon("selected.gif"); //
Add check boxes to the content pane
JCheckBox cb = new JCheckBox("C", normal);
cb.setRolloverIcon(rollover);
```

```
cb.setSelectedIcon(selected);
cb.addItemListener(this);
contentPane.add(cb);
cb = new JCheckBox("C++",
normal); cb.setRolloverIcon(rollover);
cb.setSelectedIcon(selected);
cb.addItemListener(this);
contentPane.add(cb);
cb = new JCheckBox("Java", normal);
cb.setRolloverIcon(rollover);
cb.setSelectedIcon(selected);
cb.addItemListener(this);
contentPane.add(cb);
cb = new JCheckBox("Perl", normal);
cb.setRolloverIcon(rollover);
cb.setSelectedIcon(selected);
cb.addItemListener(this);
contentPane.add(cb);
// Add text field to the content
pane jtf = new JTextField(15);
contentPane.add(jtf);
}
public void itemStateChanged(ItemEvent ie) {
JCheckBox cb = (JCheckBox)ie.getItem();
jtf.setText(cb.getText());
}
}
```

Here is the output:



Radio Buttons

Radio buttons are supported by the **JRadioButton** class, which is a concrete implementation of **AbstractButton**. Its immediate superclass is **JToggleButton**, which provides support for two-state buttons. Some of its constructors are shown here:

JRadioButton(Icon *i*)
JRadioButton(Icon *i*, boolean *state*)
JRadioButton(String *s*)
JRadioButton(String *s*, boolean *state*)
JRadioButton(String *s*, Icon *i*)
JRadioButton(String *s*, Icon *i*, boolean *state*)

Here, *i* is the icon for the button. The text is specified by *s*. If *state* is **true**, the button is initially selected. Otherwise, it is not.
Radio buttons must be configured into a group. Only one of the buttons in that group can be selected at any time. For example, if a user presses a radio button that is

in a group, any previously selected button in that group is automatically deselected.

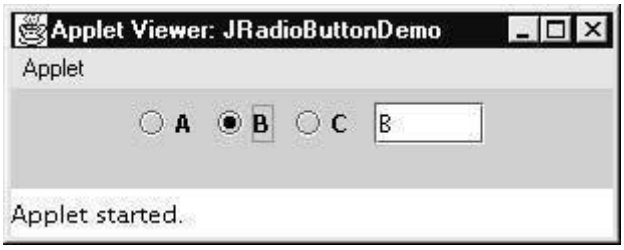
The **ButtonGroup** class is instantiated to create a button group. Its default constructor is invoked for this purpose. Elements are then added to the button group via the following method:
void add(AbstractButton *ab*)
Here, *ab* is a reference to the button to be added to the group.

Radio button presses generate action events that are handled by **actionPerformed( )**.
The **getActionCommand( )** method gets the text that is associated with a radio button and uses it to set the text field.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*; /*

<applet code="JRadioButtonDemo" width=300
height=50> </applet>
*/
public class JRadioButtonDemo extends JApplet
implements ActionListener {
JTextField tf;
public void init() {
// Get content pane
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
// Add radio buttons to content pane
JRadioButton b1 = new JRadioButton("A");
b1.addActionListener(this);
contentPane.add(b1);
JRadioButton b2 = new JRadioButton("B");
b2.addActionListener(this);
contentPane.add(b2);
JRadioButton b3 = new JRadioButton("C");
b3.addActionListener(this);
contentPane.add(b3);
// Define a button group
ButtonGroup bg = new ButtonGroup();
bg.add(b1);
bg.add(b2);
bg.add(b3);
// Create a text field and add it
// to the content pane
tf = new JTextField(5);
contentPane.add(tf);
}
public void actionPerformed(ActionEvent ae)
{ tf.setText(ae.getActionCommand());
}
}
```

Output from this applet is shown here:



Combo Boxes

Swing provides a *combo box* (a combination of a text field and a drop-down list) through the **JComboBox** class, which extends **JComponent**. A combo box normally displays one entry. However, it can also display a drop-down list that allows a user to select a different entry. You can also type your selection into the text field. Two of **JComboBox**'s constructors here:

JComboBox( )
JComboBox(Vector *v*)
Here, *v* is a vector that initializes the combo box.
Items are added to the list of choices via the **addItem( )** method, whose signature is shown here:
void addItem(Object *obj*)
Here, *obj* is the object to be added to the combo box.

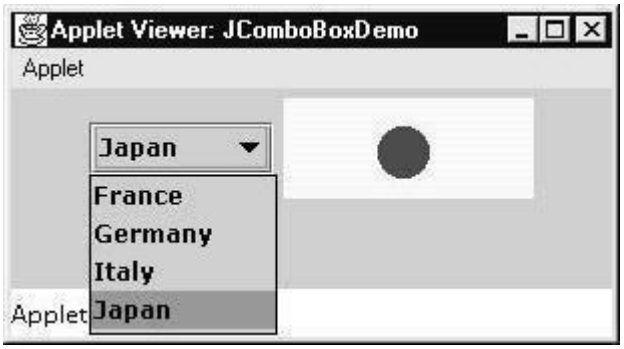
The following example contains a combo box and a label. The label displays an icon. The combo box contains entries for ―France‖,. ―Germany‖,
When a country is selected, the label is updated to display the flag for that country.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*; /*

<applet code="JComboBoxDemo" width=300
height=100> </applet>
*/
public class JComboBoxDemo extends JApplet
implements ItemListener {
JLabel jl;
ImageIcon france, germany, italy, japan;
public void init() {
// Get content pane
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
// Create a combo box and add it
// to the panel
JComboBox jc = new JComboBox();
jc.addItem("France");
jc.addItem("Germany");
jc.addItem("Italy");
jc.addItem("Japan");
jc.addItemListener(this);
contentPane.add(jc);
// Create label
```

```
jl = new JLabel(new
ImageIcon("france.gif")); contentPane.add(jl);
}
public void itemStateChanged(ItemEvent ie) {
String s = (String)ie.getItem();
jl.setIcon(new ImageIcon(s + ".gif"));
}
}
```

Output from this applet is shown here:



Tabbed Panes

A *tabbed pane* is a component that appears as a group of folders in a file cabinet. Each folder has a title. When a user selects a folder, its contents become visible. Only one of the folders may be selected at a time. Tabbed panes are commonly used for setting configuration options.

Tabbed panes are encapsulated by the **JTabbedPane** class, which extends **JComponent**. We will use its default constructor. Tabs are defined via the following method:

void addTab(String *str*, Component *comp*)

Here, *str* is the title for the tab, and *comp* is the component that should be added to the tab. Typically, a **JPanel** or a subclass of it is added.

The general procedure to use a tabbed pane in an applet is outlined here:

1. Create a **JTabbedPane** object.
2. Call **addTab( )** to add a tab to the pane. (The arguments to this method define the title of the tab and the component it contains.)
3. Repeat step 2 for each tab.
4. Add the tabbed pane to the content pane of the applet.

Scroll Panes

A *scroll pane* is a component that presents a rectangular area in which a componentmay be viewed. Horizontal and/or vertical scroll bars may be provided if necessary.

Scroll panes are implemented in Swing by the **JScrollPane** class, which extends**JComponent**. Some of its constructors are shown here:

JScrollPane(Component *comp*) JScrollPane(int *vsb*, int *hsb*) JScrollPane(Component *comp*, int *vsb*, int *hsb*)

Here, *comp* is the component to be added to the scroll pane. *vsb* and *hsb* are **int**constants that define when vertical and horizontal scroll bars for this scroll pane areshown. These constants are defined by the **ScrollPaneConstants** interface. Some
examples of these constants are described as follows:
Constant Description
HORIZONTAL_SCROLLBAR_ALWAYS Always provide horizontal scroll bar
HORIZONTAL_SCROLLBAR_AS_NEEDED Provide horizontal scroll bar, if needed
VERTICAL_SCROLLBAR_ALWAYS Always provide vertical scroll bar
VERTICAL_SCROLLBAR_AS_NEEDED Provide vertical scroll bar, if needed

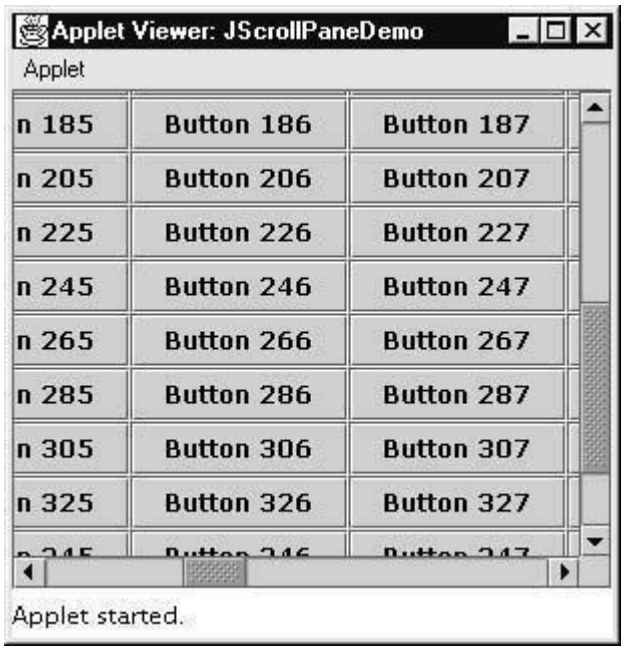
Here are the steps that you should follow to use a scroll pane in an applet:

1. Create a **JComponent** object.
2. Create a **JScrollPane** object. (The arguments to the constructor specify the component and the policies for vertical and horizontal scroll bars.)
3. Add the scroll pane to the content pane of the applet.

The following example illustrates a scroll pane. First, the content pane of the **JApplet** object is obtained and a border layout is assigned as its layout manager. Next, a **JPanel** object is created and four hundred buttons are added to it, arranged into twenty columns. The panel is then added to a scroll pane, and the scroll pane is added to the content pane. This causes vertical and horizontal scroll bars to appear. You can use the scroll bars to scroll the buttons into view.

```
import java.awt.*;
import javax.swing.*;
/*
<applet code="JScrollPaneDemo" width=300 height=250>
</applet>
*/
public class JScrollPaneDemo extends JApplet
{ public void init() {
// Get content pane
Container contentPane = getContentPane();
contentPane.setLayout(new BorderLayout());
// Add 400 buttons to a panel
JPanel jp = new JPanel();
jp.setLayout(new GridLayout(20, 20));
int b = 0;
for(int i = 0; i < 20; i++) {
for(int j = 0; j < 20; j++) {
jp.add(new JButton("Button " + b));
++b;
}
}
// Add panel to a scroll pane
int v = ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
int h = ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;
JScrollPane jsp = new JScrollPane(jp, v, h);
// Add scroll pane to the content pane
contentPane.add(jsp, BorderLayout.CENTER);
}
}
```

Output from this applet is shown here:



Trees

A *tree* is a component that presents a hierarchical view of data. A user has the ability to expand or collapse individual subtrees in this display. Trees are implemented in Swing by the **JTree** class, which extends **JComponent**. Some of its constructors are shown here:

JTree(Hashtable *ht*)
JTree(Object *obj*[ ])
JTree(TreeNode *tn*)
JTree(Vector *v*)

The first form creates a tree in which each element of the hash table *ht* is a child node.

Each element of the array *obj* is a child node in the second form. The tree node *tn* is the root of the tree in the third form. Finally, the last form uses the elements of vector *v* as child nodes.

A **JTree** object generates events when a node is expanded or collapsed. The **addTreeExpansionListener( )** and **removeTreeExpansionListener( )** methods allow listeners to register and unregister for these notifications. The signatures of these methods are shown here:
void addTreeExpansionListener(TreeExpansionListener *tel*)
void removeTreeExpansionListener(TreeExpansionListener *tel)*

Here, *tel* is the listener object.

The **getPathForLocation( )** method is used to translate a mouse click on a specific point of the tree to a tree path. Its signature is shown here:
TreePath getPathForLocation(int *x*, int *y*)
Here, *x* and *y* are the coordinates at which the mouse is clicked. The return value is a **TreePath** object that encapsulates information about the tree node that was selected by the user.

Tables

A *table* is a component that displays rows and columns of data. You can drag the cursor on column boundaries to resize columns. You can also drag a column to a new position. Tables are implemented by the **JTable** class, which extends **JComponent**.

One of its constructors is shown here:

JTable(Object *data*[ ][ ], Object *colHeads*[ ])

Here, *data* is a two-dimensional array of the information to be presented, and *colHeads* is a one-dimensional array with the column headings.

Here are the steps for using a table in an applet:

1. Create a **JTable** object.
2. Create a **JScrollPane** object. (The arguments to the constructor specify the table and the policies for vertical and horizontal scroll bars.)
3. Add the table to the scroll pane.
4. Add the scroll pane to the content pane of the applet.

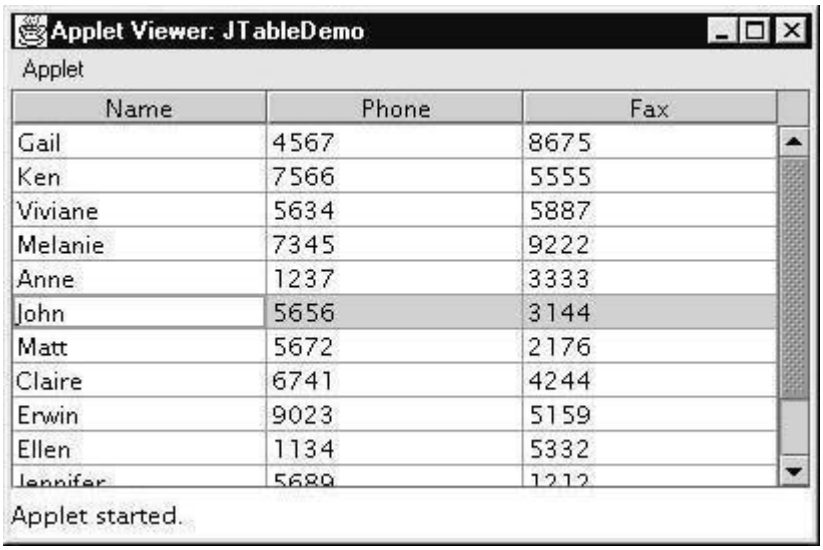
The following example illustrates how to create and use a table. The content pane of the **JApplet** object is obtained and a border layout is assigned as its layout manager.A one-dimensional array of strings is created for the column headings. This table has three columns. A two-dimensional array of strings is created for the table cells. You can see that each element in the array is an array of three strings. These arrays are passed to the **JTable** constructor. The table is added to a scroll pane and then the scroll pane is added to the content pane.

```
import java.awt.*;
import javax.swing.*;
/*
<applet code="JTableDemo" width=400
height=200> </applet>
*/
public class JTableDemo extends JApplet {

public void init() {
// Get content pane
Container contentPane = getContentPane();
// Set layout manager
contentPane.setLayout(new BorderLayout());
// Initialize column headings
final String[] colHeads = { "Name", "Phone", "Fax"
}; // Initialize data
final Object[][] data = {
{ "Gail", "4567", "8675" },
{ "Ken", "7566", "5555" },
{  "Viviane", "5634", "5887"   },
{ "Melanie", "7345", "9222"   },
{ "Anne", "1237", "3333" },
{ "John", "5656", "3144" },
{ "Matt", "5672", "2176" }, {
"Claire", "6741", "4244" }, {
"Erwin", "9023", "5159" }, {
"Ellen", "1134", "5332" },
{ "Jennifer", "5689", "1212" },
{ "Ed", "9030", "1313" },
```

```
{ "Helen", "6751", "1415"
} };
// Create the table
JTable table = new JTable(data,
colHeads); // Add table to a scroll pane
int v = ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
int h = ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;
JScrollPane jsp = new JScrollPane(table, v, h);
// Add scroll pane to the content pane
contentPane.add(jsp, BorderLayout.CENTER);
}
}
```

Here is the output:



Event Handling is at the core of successful applet programming. Most events to which your applet will respond are generated by the user. These events are passed to your applet in a variety of ways, with the specific method depending upon the actual event. There are several types of events. The most commonly handled events are those generated by the mouse, the keyboard, and various controls, such as a push button. Events are supported by the **java.awt.event** package

Events

In the delegation model, an *event* is an object that describes a state change in a source. It can be generated as a consequence of a person interacting with the elements in a graphical user interface. Some of the activities that cause events to be generated are pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse. Many other user operations could also be cited as examples. Events may also occur that are not directly caused by interactions with a user interface. For example, an event may be generated when a timer expires, a counter exceeds a value, a software or hardware failure occurs, or an operation is completed. You are free to define events that are appropriate for your application.

Event Sources

A *source* is an object that generates an event. This occurs when the internal state of that object changes in some way. Sources may generate more than one type of event. A source must register listeners in order for the listeners to receive notifications about a specific type of event. Each type of event has its own registration method.

Here is the general form:
public void add*Type*Listener(*Type*Listener *el*)

Here, *Type* is the name of the event and *el* is a reference to the event listener. For example, the method that registers a keyboard event listener is called **addKeyListener( )**. The method that registers a mouse motion listener is called **addMouseMotionListener( )**. When an event occurs, all registered listeners are notified and receive a copy of the event object. This is known as *multicasting* the event. In all cases, notifications are sent only to listeners that register to receive them. Some sources may allow only one listener to register. The general form of such a method is this:

public void add*Type*Listener(*Type*Listener *el*)
throws java.util.TooManyListenersException

Here, *Type* is the name of the event and *el* is a reference to the event listener. When such an event occurs, the registered listener is notified. This is known as *unicasting* the event.

A source must also provide a method that allows a listener to unregister an interest in a specific type of event.

The general form of such a method is this:
public void remove*Type*Listener(*Type*Listener *el*)

Here, *Type* is the name of the event and *el* is a reference to the event listener. For example, to remove a keyboard listener, you would call **removeKeyListener( )**.

The methods that add or remove listeners are provided by the source that generates events. For example, the **Component** class provides methods to add and remove keyboard and mouse event listeners.

Event Classes

The classes that represent events are at the begin our study of event handling with a tour of the event classes. As you will see, they provide a consistent, easy-to-use means of encapsulating events.

At the root of the Java event class hierarchy is **EventObject**, which is in **java.util**. It is the superclass for all events. Its one constructor is shown here:

EventObject(Object *src*)

Here, *src* is the object that generates this event.

EventObject contains two methods: **getSource( )** and **toString( )**. The **getSource( )** method returns the source of the event. Its general form is shown here:

Object getSource( )

As expected, **toString( )** returns the string equivalent of the event.

The class **AWTEvent**, defined within the **java.awt** package, is a subclass of**EventObject**. It is the superclass (either directly or indirectly) of all AWT-based events used by the delegation event model. Its **getID( )** method can be used to determine the type of the event. The signature of

this method is shown here:

int getID( )

Additional details about **AWTEvent** are provided at the end of Chapter 22. At this point, it is important to know only that all of the other classes discussed in this section are subclasses of **AWTEvent**.

To summarize:

- **EventObject** is a superclass of all events.
- **AWTEvent** is a superclass of all AWT events that are handled by the delegation event model.

The package **java.awt.event** defines several types of events that are generated by various user interface elements. Table 20-1 enumerates the most important of these event classes and provides a brief description of when they are generated. The most commonly used constructors and methods in each class are described in the following sections.

Event Listeners

A *listener* is an object that is notified when an event occurs. It has two major requirements. First, it must have been registered with one or more sources to receive notifications about specific types of events. Second, it must implement methods to receive and process these notifications.

The methods that receive and process events are defined in a set of interfaces found in **java.awt.event**. For example, the **MouseMotionListener** interface defines two methods to receive notifications when the mouse is dragged or moved. Any object may receive and process one or both of these events if it provides an implementation of this interface.

The Delegation Event Model

The modern approach to handling events is based on the *delegation event model*, which defines standard and consistent mechanisms to generate and process events. Its conceptis quite simple: a *source* generates an event and sends it to one or more *listeners*. In thisscheme, the listener simply waits until it receives an event. Once received, the listener processes the event and then returns. The advantage of this design is that the applicationlogic that processes events is cleanly separated from the user interface logic that generates those events. A user interface element is able to ―delegate‖ to theaseparateprocessingpieceofcode. of an event
In the delegation event model, listeners must register with a source in order to receive an event notification. This provides an important benefit: notifications are sent only to listeners that want to receive them. This is a more efficient way to handle events than the design used by the old Java 1.0 approach. Previously, an event was propagated up the containment hierarchy until it was handled by a component. This required components to receive events that they did not process, and it wasted valuable time. The delegation event model eliminates this overhead.

The MouseEvent Class

There are eight types of mouse events. The **MouseEvent** class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED The user clicked the mouse.
MOUSE_DRAGGED The user dragged the mouse.
MOUSE_ENTERED The mouse entered a component.
MOUSE_EXITED The mouse exited from a component.
MOUSE_MOVED The mouse moved.
MOUSE_PRESSED The mouse was pressed.
MOUSE_RELEASED The mouse was released.
MOUSE_WHEEL The mouse wheel was moved (Java 2, v1.4).

MouseEvent is a subclass of **InputEvent**. Here is one of its constructors.
MouseEvent(Component *src*, int *type*, long *when*, int *modifiers*,

int *x*, int *y*, int *clicks*, boolean *triggersPopup*)

Here, *src* is a reference to the component that generated the event. The type of the event is specified by *type*. The system time at which the mouse event occurred is passed in *when*. The *modifiers* argument indicates which modifiers were pressed when a mouse event occurred. The coordinates of the mouse are passed in *x* and *y*. The click count is passed in *clicks*. The *triggersPopup* flag indicates if this event causes a pop-up menu to appear on this platform. Java 2, version 1.4 adds a second constructor which also allows the button that caused the event to be specified.

The most commonly used methods in this class are **getX( )** and **getY( )**. These return the X and Y coordinates of the mouse when the event occurred. Their forms are shown here:

int getX( )
int getY( )

Alternatively, you can use the **getPoint( )** method to obtain the coordinates of the mouse.

It is shown here:
Point getPoint( )
It returns a **Point** object that contains the X, Y coordinates in its integer members: **x** and **y**.
The **translatePoint( )** method changes the location of the event. Its form is shown here:
void translatePoint(int *x*, int *y*)

Here, the arguments *x* and *y* are added to the coordinates of the event.
The **getClickCount( )** method obtains the number of mouse clicks for this event.
Its signature is shown here:

int getClickCount( )

The **isPopupTrigger( )** method tests if this event causes a pop-up menu to appear on this platform. Its form is shown here:
boolean isPopupTrigger( )

Java 2, version 1.4 added the **getButton( )** method, shown here.
int getButton( )
It returns a value that represents the button that caused the event. The return value will be one of these constants defined by **MouseEvent**.
NOBUTTON BUTTON1 BUTTON2 BUTTON3
The **NOBUTTON** value indicates that no button was pressed or released

Handling Keyboard Events

To handle keyboard events, you use the same general architecture as that shown in the mouse event example in the preceding section. The difference, of course, is that you will be implementing the **KeyListener** interface.

Before looking at an example, it is useful to review how key events are generated.

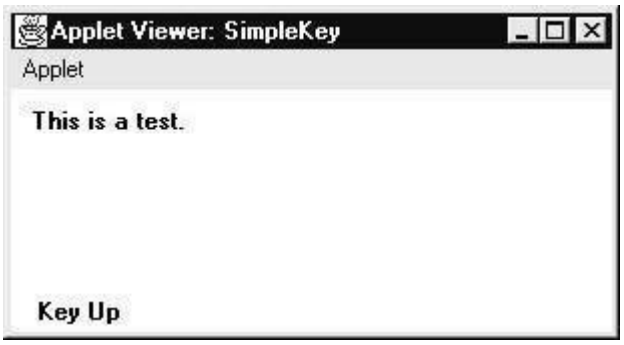
When a key is pressed, a **KEY_PRESSED** event is generated. This results in a call to the **keyPressed( )** event handler. When the key is released, a **KEY_RELEASED** event is generated and the **keyReleased( )** handler is executed. If a character is generated bythe keystroke, then a **KEY_TYPED** event is sent and the **keyTyped( )** handler is invoked.Thus, each time the user presses a key, at least two and often three events are generated. If all you care about are actual characters, then you can ignore the information passed by the key press and release events. However, if your program needs to handle special keys, such as the arrow or function keys, then it must watch for them through the **keyPressed( )** handler.

There is one other requirement that your program must meet before it can process keyboard events: it must request input focus. To do this, call **requestFocus( )**, which is defined by **Component**. If you don't, then yourkeyboard eventsprogram. will n

```
// Demonstrate the key event handlers.
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="SimpleKey" width=300 height=100>
</applet>
*/
public class SimpleKey extends Applet
implements KeyListener {
String msg = "";
int X = 10, Y = 20; // output
coordinates public void init() {
addKeyListener(this);
requestFocus(); // request input focus
}
public void keyPressed(KeyEvent ke)
{ showStatus("Key Down");
}
public void keyReleased(KeyEvent ke)
{ showStatus("Key Up");
}
public void keyTyped(KeyEvent ke)
{ msg += ke.getKeyChar();
repaint();
}
// Display keystrokes.
public void paint(Graphics g)
{ g.drawString(msg, X, Y);
}
}
```

Sample output is shown here:

If you want to handle the special keys, such as the arrow or function keys, you need to respond to them within the **keyPressed( )** handler. They are not available through **keyTyped( )**. To identify the keys, you use their virtual key codes. For example, the next applet outputs the name of a few of the special keys:



Adapter Classes

Java provides a special feature, called an *adapter class*, that can simplify the creation of event handlers in certain situations. An adapter class provides an empty implementation of all methods in an event listener interface. Adapter classes are useful when you want to receive and process only some of the events that are handled by a particular event listener interface. You can define a new class to act as an event listener by extending one of the adapter classes and implementing only those events in which you are interested.

For example, the **MouseMotionAdapter** class has two methods, **mouseDragged( )** and **mouseMoved( )**. The signatures of these empty methods are exactly as defined in the **MouseMotionListener** interface. If you were interested in only mouse drag events, then you could simply extend **MouseMotionAdapter** and implement **mouseDragged( )**. The empty implementation of **mouseMoved( )** would handle the mouse motion events for you. Table 20-4 lists the commonly used adapter classes in **java.awt.event** and notes the interface that each implements.

The following example demonstrates an adapter. It displays a message in the status bar of an applet viewer or browser when the mouse is clicked or dragged. However, all other mouse events are silently ignored. The program has three classes. **AdapterDemo** extends **Applet**. Its **init( )** method creates an instance of **MyMouseAdapter** and registers that object to receive notifications of mouse events. It also creates an instance of **MyMouseMotionAdapter** and registers that object to receive notifications of mouse motion events. Both of the constructors take a reference to the applet as an argument. **MyMouseAdapter** implements the **mouseClicked( )** method. The other mouse events are silently ignored by code inherited from the **MouseAdapter** class.

MyMouseMotionAdapter implements the **mouseDragged( )** method. The other mouse motion event is silently ignored by code inherited from the **MouseMotionAdapter** class.

Adapter Class	**Listener Interface**
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener

FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	
MouseMotionListener	
WindowAdapter	WindowListener

Demonstrate an adapter.

```
import java.awt.*;
import java.awt.event.*;
import java.applet.*;
/*
<applet code="AdapterDemo" width=300 height=100>
</applet>
*/
public class AdapterDemo extends Applet
{ public void init() {
addMouseListener(new MyMouseAdapter(this));
addMouseMotionListener(new MyMouseMotionAdapter(this));
}
}
class MyMouseAdapter extends MouseAdapter
{ AdapterDemo adapterDemo;
public MyMouseAdapter(AdapterDemo adapterDemo) {

this.adapterDemo = adapterDemo;

}

// Handle mouse clicked.
public void mouseClicked(MouseEvent me) {
adapterDemo.showStatus("Mouse clicked");
}
}
class MyMouseMotionAdapter extends MouseMotionAdapter {
AdapterDemo adapterDemo;
public MyMouseMotionAdapter(AdapterDemo adapterDemo) {
this.adapterDemo = adapterDemo;
}

// Handle mouse dragged.
public void mouseDragged(MouseEvent me) {
adapterDemo.showStatus("Mouse dragged");
}
}
```